Практическая работа №4

Автоматизация тестирования веб-приложений с использованием библиотеки Selenium и фреймворка Pytest.

Цель работы

Изучить возможности библиотеки Selenium для автоматизации действий в веб-браузере. Научиться открывать веб-страницы, находить элементы (по ID, CSS-селектору, имени тега), взаимодействовать с ними (вводить текст, нажимать кнопки) и извлекать данные. Освоить написание автотестов с помощью Pytest для проверки корректности полученных данных с сайта и демонстрации падения теста при неверных (подмененных) данных.

В ходе работы была выполнена автоматизация получения расписания с сайта https://api.nntu.ru/. Для этого были реализованы два файла:

schedule_parser - скрипт на Selenium, который:

- запускает браузер Google Chrome;
- открывает сайт с расписанием;
- находит и извлекает необходимый текст со страницы (с предварительной обработкой возможных таймаутов и ошибок сертификатов сайта).

test_schedule - тесты на Pytest, которые:

- используют фикстуру для получения реального текста расписания;
- проверяют наличие нужных дисциплин в полученном расписании (правильный результат);
- демонстрируют падение теста при подмене расписания на фейковое (неправильный результат).

schedule_parser

```
from selenium import webdriver

from selenium.webdriver.common.by import By

from selenium.webdriver.support.ui import WebDriverWait

from selenium.webdriver.support import expected_conditions as EC

import time

def get_schedule_text():

options = webdriver.ChromeOptions()

# options.add_argument('--headless') #

\driver = webdriver.Chrome(options=options)

try:
```

```
driver.get("https://api.nntu.ru/raspisanie")
wait = WebDriverWait(driver, 15)
wait.until(EC.text_to_be_present_in_element((By.TAG_NAME, "body"), "Понедельник"))
time.sleep(2) #
page_text = driver.find_element(By.TAG_NAME, "body").text
return page_text
finally:
driver.quit()
if __name__ == "__main__":
print(get_schedule_text()[:500])
```

```
test_schedule
import pytest
from schedule_parser import get_schedule_text
@pytest.fixture(scope="module")
def real_schedule():
print("\nЗагрузка расписания с сайта...")
text = get_schedule_text()
return text
@pytest.mark.basic
def test_schedule_matches_expected(real_schedule):
assert "Эксплуатация и модификация" in real_schedule
assert "Информационно-коммуникационные системы" in real_schedule
assert "Надежность и отказоустойчивость" in real_schedule
assert "Теория цифровой обработки" in real_schedule
@pytest.mark.exception
def test_wrong_schedule_fails(real_schedule):
fake_schedule = real_schedule.replace("Английский", "Химия")
```

assert "Английский" in fake_schedule

@pytest.mark.skip(reason="Этот тест просто пропускается для демонстрации")

def test_skip_example():

assert True



Рис. 1. Процесс установки библиотеки Selenium



Рис. 2. Завершение установки пакетов.

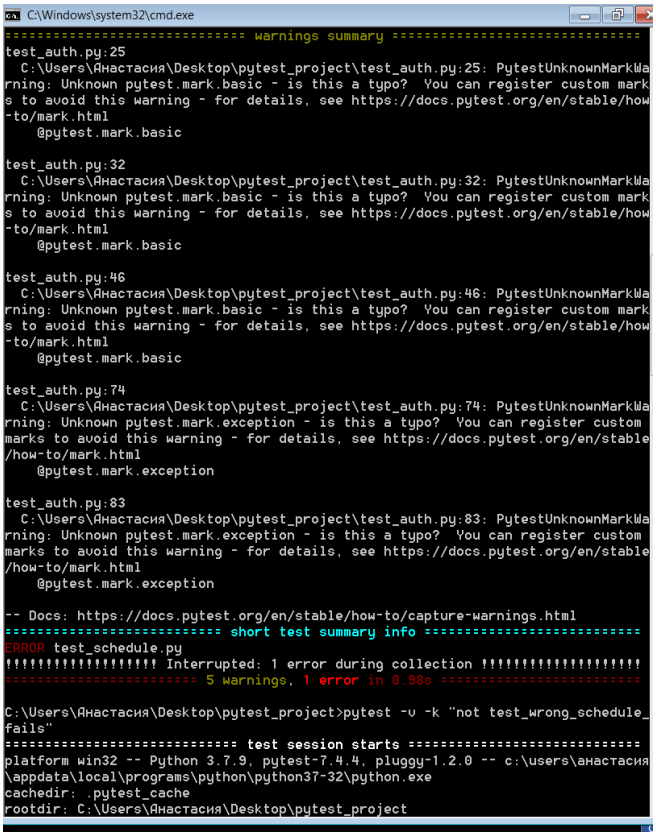


Рис. 3. Запуск теста test_schedule

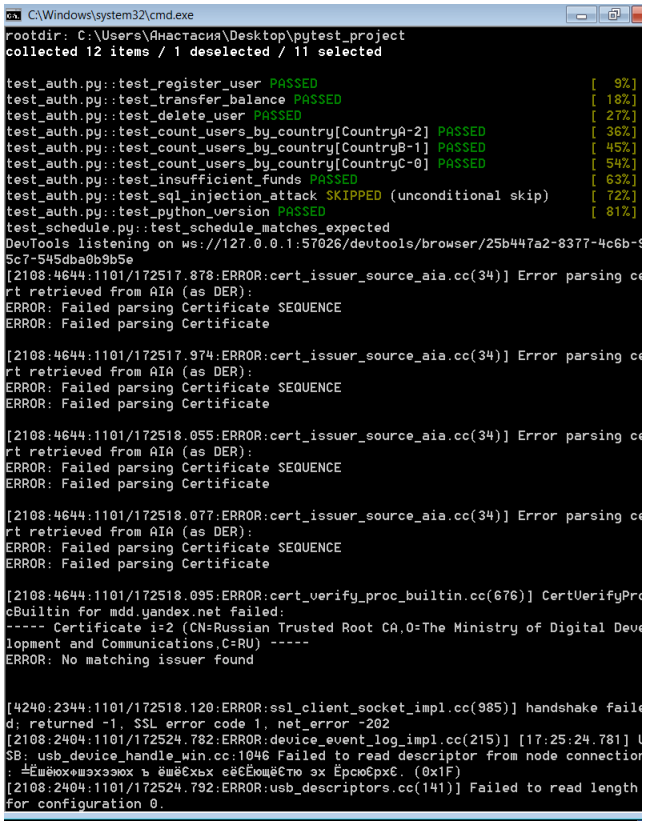


Рис. 4. Результат запуска тестов

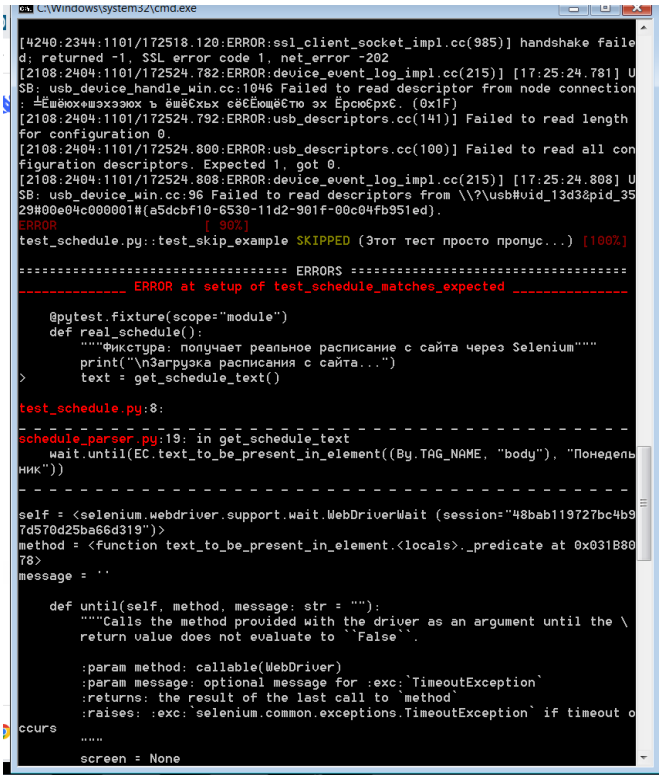


Рис. 5. Стек ошибки TimeoutException

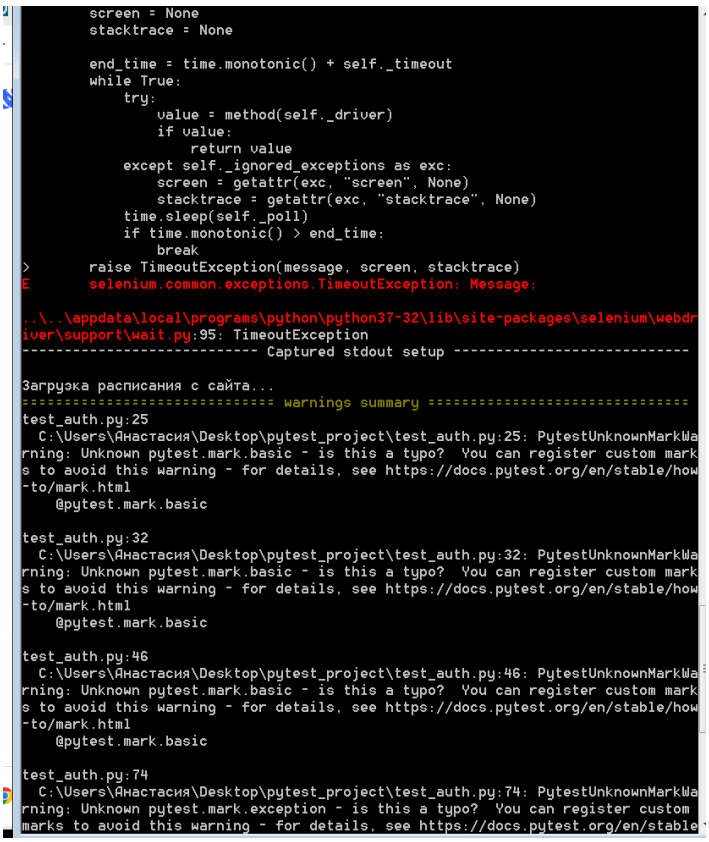


Рис. 6. Стек ошибки TimeoutException

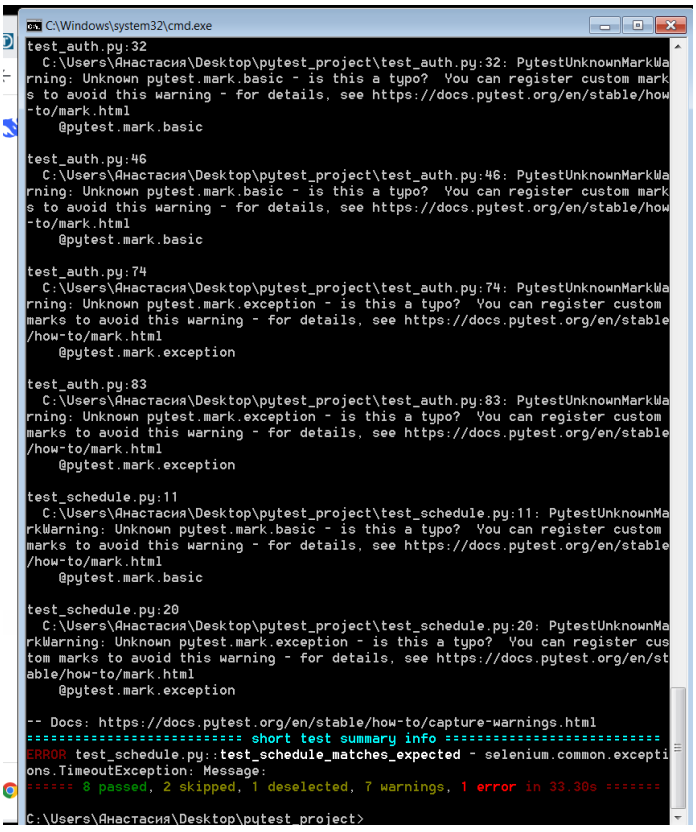


Рис. 7. Демонстрация работы Selenium

Тест на получение данных с сайта не прошел из-за блокировки парсинга сайтом (ошибка сертификата/таймаута), однако Selenium работает верно.

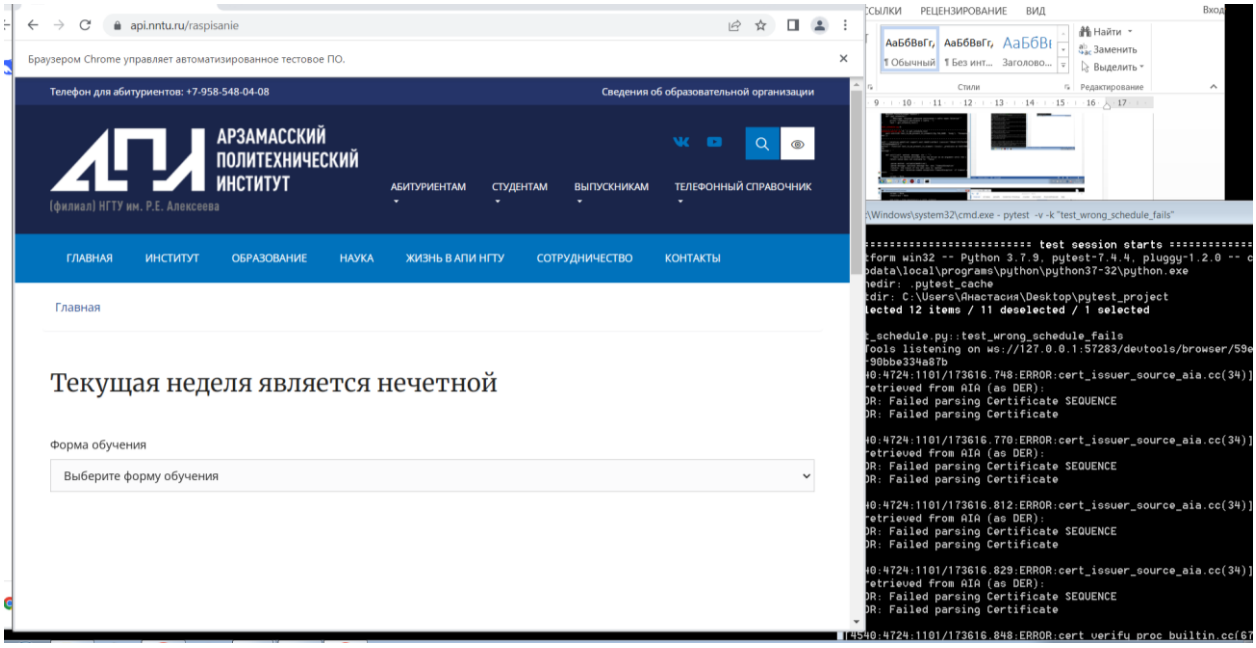


Рис. 8. Демонстрация работы Selenium

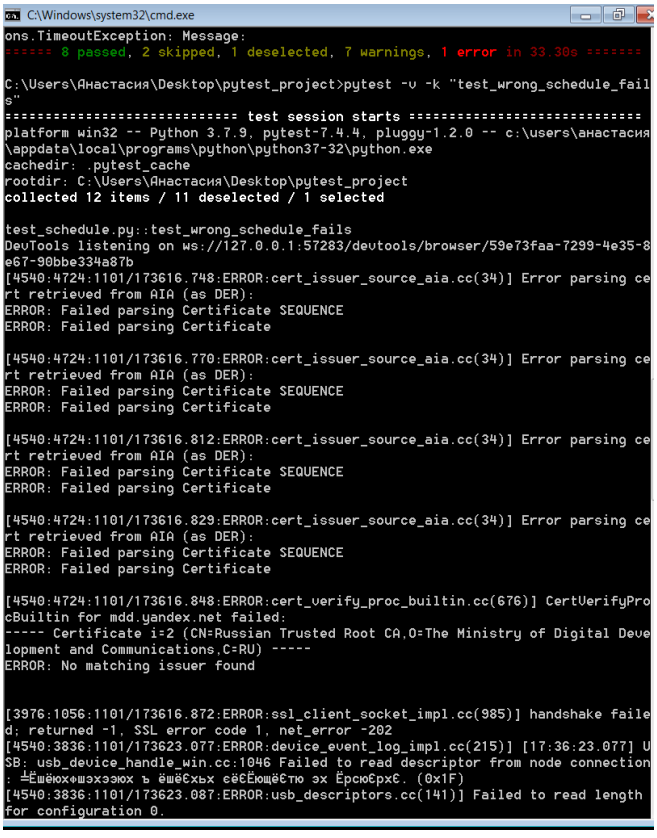


Рис. 9. Запуск теста test_wrong_schedule_fails

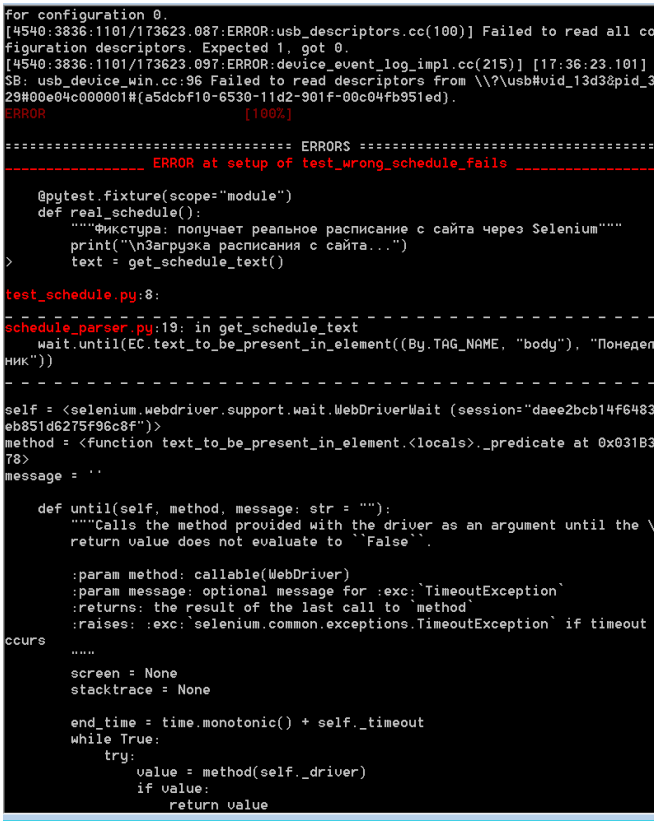


Рис. 10. Вывод ошибки TimeoutException

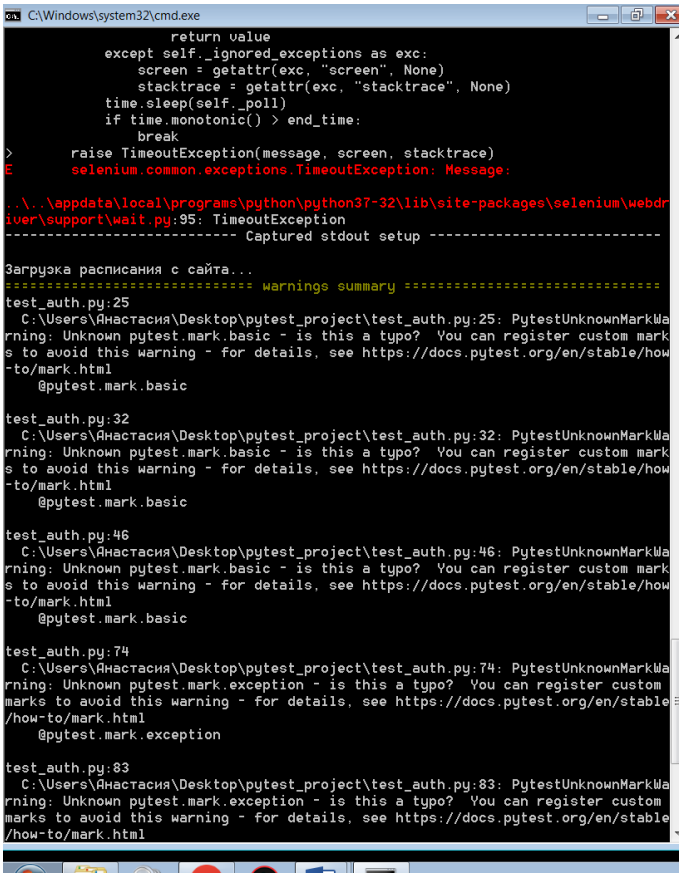


Рис. 11. Запуск теста с подменой данных.

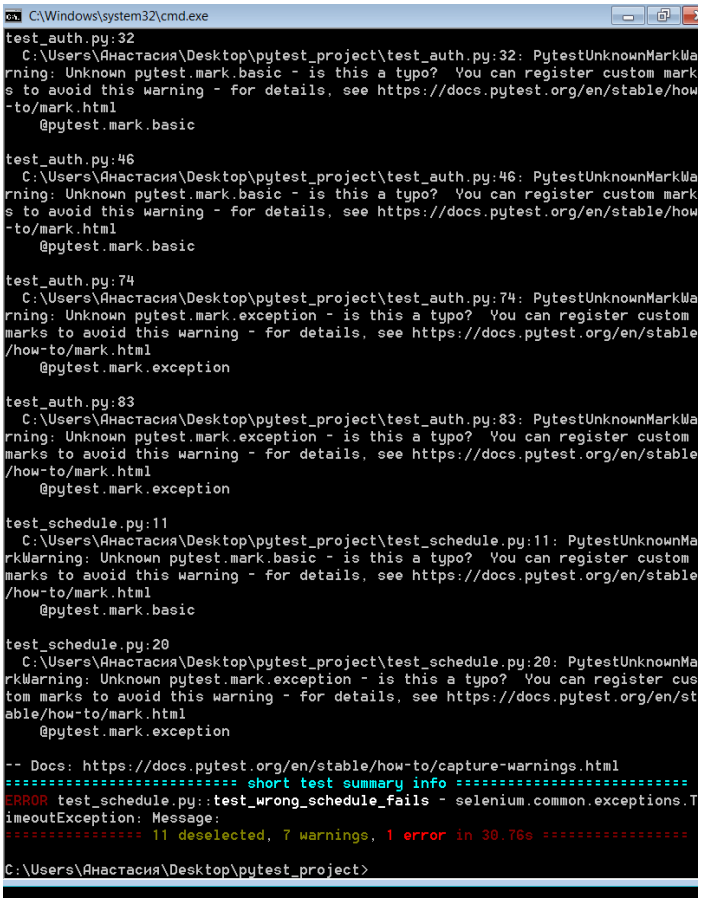


Рис. 12. Итоговый результат выполнения тестов

Тест на подмену расписания падает с AssertionError. Это показывают, что тесты работают и ловят ошибки.

Вывод

В ходе выполнения лабораторной работы были получены практические навыки автоматизации тестирования веб-приложений с помощью библиотеки Selenium. Были освоены методы поиска элементов на странице (по ID, CSS-селекторам, имени тега) и взаимодействия с ними (ввод текста, нажатие кнопок). Продемонстрировано, что при подмене данных тест выявляет ошибку и завершается с FAILED.